



Rapport du mini-projet LU3IN025

Affectation des étudiants en Master Informatique

Helder Brito (21304177) et O'nel Hounnouvi (21315612)

Avril 2026 – Mai 2026

Introduction

Ce projet modélise le processus d'affectation des étudiants aux 10 parcours du Master Informatique de Sorbonne Université. L'admission reposant sur des préférences bilatérales (les vœux des candidats et l'évaluation de leurs dossiers par les responsables), l'objectif de cette étude est d'implémenter et d'analyser l'algorithme de Gale-Shapley afin de garantir une répartition stable et optimale.

1 Problème et affectation

1.1 Gale-Shapley : Côté étudiants

1.1.1 Choix des structures de données pour l'implémentation

- **Étudiants libres (etu_libres) : File (Queue)**

Implémentée via une `deque` (Double-Ended Queue), elle permet de stocker les étudiants n'ayant pas encore été affectés à un master.

- *Extraction du premier arrivé* : $\mathcal{O}(1)$

- *Ajout d'un étudiant rejeté* : $\mathcal{O}(1)$

- **Progression des propositions (prochain_voeu) : Tableau**

Au lieu de faire un pop sur la liste de préférences d'un étudiant (et donc la détruire), ce tableau stocke simplement l'indice de sa prochaine proposition. Ainsi, `prochain_voeu[i]` indique l'indice du prochain master à proposer pour l'étudiant e_i et est incrémenté à chaque proposition. L'instruction `PrefEtu[i][prochain_voeu[i]]` permet d'obtenir le master ciblé sans modifier la liste de préférences.

- *Lecture et Incrémentation* : $\mathcal{O}(1)$

- **Position de l'étudiant i dans un parcours j (classement) : Matrice**

Une matrice `classement` calculée en amont en inversant la matrice de préférences des masters (`PrefSpe`). Ainsi, `classement[i][j]` stocke le rang de l'étudiant i pour le master j . Cela évite au master de parcourir sa liste de préférences en $\mathcal{O}(n)$ à chaque proposition.

- *Recherche du rang de l'étudiant* : $\mathcal{O}(1)$

- **Gestion des affectations (AFFECTATIONS) : Tas Min (Min-Heap)**

Pour chaque master, la liste des étudiants acceptés est maintenue dans une file de priorité (Tas binaire). En insérant l'opposé du rang, le pire étudiant (le plus grand rang, donc le plus petit nombre négatif) est toujours à la racine du tas. Si c_j est la capacité du master j , le tas permet de:

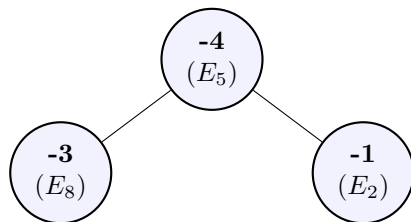
- *Identifier le pire étudiant (Racine)* : $\mathcal{O}(1)$

- *Insérer un nouvel étudiant* : $\mathcal{O}(\log c_j)$

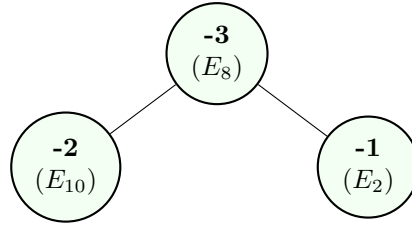
- *Remplacer le pire étudiant* : $\mathcal{O}(\log c_j)$

Fonctionnement du Tas Min (Exemple)

Imaginons un parcours de capacité $c = 3$. Il a actuellement accepté les étudiants E_2 (rang 1), E_8 (rang 3) et E_5 (rang 4). La racine contient le pire candidat à éjecter en cas de meilleure proposition.



Si un étudiant E_{10} de rang 2 postule, le master regarde la racine (-4). Puisque $-2 > -4$, E_{10} remplace E_5 , et le tas se rééquilibre automatiquement en $\mathcal{O}(\log c)$.



1.1.2 Complexité de l'algorithme

L'algorithme de Gale-Shapley garantit que chaque étudiant propose à chaque master au plus une fois. Avec n étudiants et m masters, le nombre total de propositions est au maximum $n \times m$.

- *Boucle principale (Propositions)* : $\mathcal{O}(n \times m)$ dans le pire des cas.
- *Gestion des affectations* : À chaque proposition, l'étudiant est évalué par le master. Dans le pire des cas, cette opération coûte $\mathcal{O}(\log c_j)$.

Ainsi, la complexité temporelle pire cas de l'algorithme est de $\mathcal{O}(n \times m \log c_{\max})$, où $c_{\max}$ est la plus grande capacité parmi tous les masters.

Remarque : En pratique, les capacités c_j des masters sont des valeurs petites et fixes, indépendantes du nombre total de candidats n . Le terme $\log c_{\max}$ agit donc comme une constante $\mathcal{O}(1)$ et la complexité effective est $\mathcal{O}(n \times m)$.

1.2 Gale-Shapley : Côté parcours

Dans cette variante, les rôles sont inversés : ce sont les parcours qui font des propositions aux étudiants.

Ce changement de paradigme simplifie grandement les structures de données nécessaires. Puisque chaque étudiant ne peut accepter qu'une seule affectation (capacité de 1), la gestion d'une file de priorité (Tas Min) du côté de la réception des offres n'est plus requise. Une simple comparaison d'entiers suffit.

Voici les structures de données retenues pour garantir une exécution optimale :

- **Parcours libres (parcours_libres) : File (Queue)**
Implémentée via une `deque`. Elle stocke les indices des parcours qui ont encore des places disponibles et qui n'ont pas encore épuisé leur liste de candidats. L'extraction et l'ajout se font en $\mathcal{O}(1)$.
- **Progression des propositions (prochain_voeu) : Tableau**
Un tableau de taille m stockant un pointeur pour chaque parcours. Il indique l'indice i du prochain étudiant à démarcher dans la liste `PrefSpe[i]`. L'incréméntation se fait en $\mathcal{O}(1)$.
- **Gestion des affectations (affectation_etu) : Tableau 1D**
Ce tableau de taille n vient remplacer la structure de Tas Min de la variante précédente. `affectation_etu[i]` stocke simplement l'indice du master actuellement retenu par l'étudiant e_i (initialisé à -1). En cas de meilleure offre, cette valeur est écrasée en $\mathcal{O}(1)$.
- **Comparaison des candidatures (classement_etu) : Matrice**
Pour que l'étudiant puisse évaluer la nouvelle proposition quasi instantanément, cette matrice pré-calculée fournit le rang d'un master donné pour un étudiant précis en $\mathcal{O}(1)$.

Complexité : La boucle principale s'exécute tant qu'il y a des parcours libres. Dans le pire des cas, tous les parcours feront une offre à tous les étudiants de leur liste. Le nombre maximum de propositions reste donc borné par $\mathcal{O}(n \times m)$. Puisque chaque traitement (vérification du rang, mise à jour de l'affectation, réinsertion dans la file) s'effectue en temps constant $\mathcal{O}(1)$, la complexité temporelle théorique de cette variante est strictement $\mathcal{O}(n \times m)$.

1.3 Affectations obtenues sur les fichiers tests et stabilité

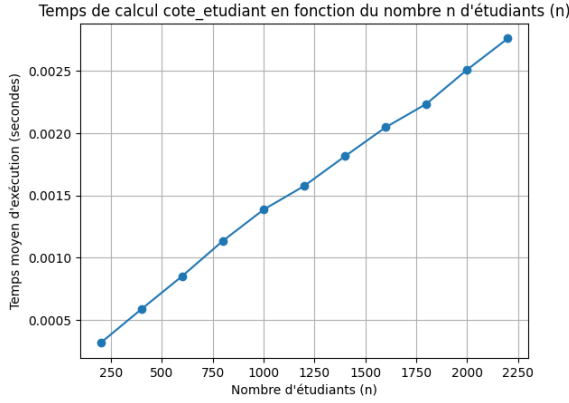
- On obtient les mêmes affectations en appliquant l'algorithme de Gale-Shapley, que ce soit du côté des étudiants ou du côté des parcours, à l'ordre d'affectation près.
- La stabilité de l'affectation est vérifiée par notre fonction `liste_paires_instables` qui retourne une liste vide, indiquant qu'il n'existe aucune paire instable.

Table 1: Côté étudiants et côté parcours

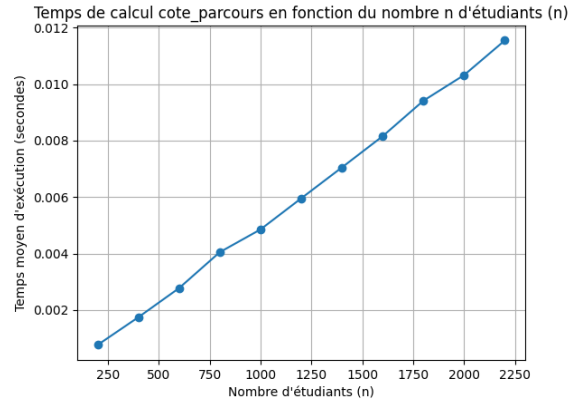
Parcours	0	1	2	3	4	5	6	7	8	9
Étudiants	E_5, E_{12}	E_4	E_9	E_8	E_{10}, E_{11}	E_0	E_1	E_7	E_6	E_2, E_3

2 Évolution du temps de calcul

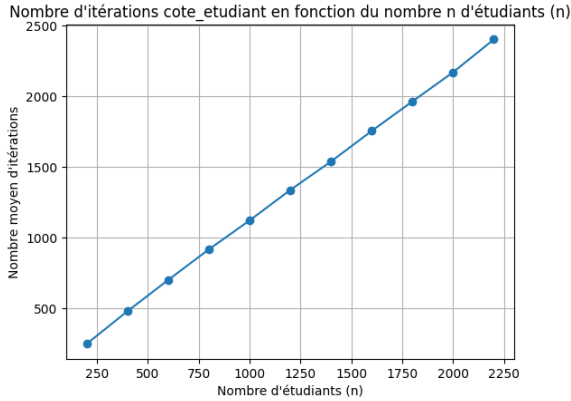
Voici les graphiques obtenus pour les différentes configurations d'affectation (côté étudiants et côté parcours) en fonction du nombre d'étudiants n (le nombre de parcours m étant fixé à 10).



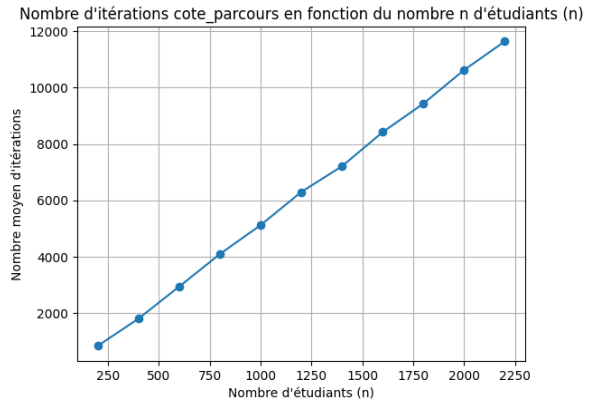
(a) Analyse côté Étudiant



(b) Analyse côté Parcours

Figure 1: Évolution du temps de calcul en fonction de n 

(a) Analyse côté Étudiant



(b) Analyse côté Parcours

Figure 2: Nombre moyen d'itérations pour atteindre la convergence

Comme le mettent en évidence ces graphiques, le temps d'exécution et le nombre d'itérations croissent linéairement en fonction du nombre d'étudiants n . Nous observons donc une complexité empirique en $\mathcal{O}(n)$ pour nos deux variantes de l'algorithme de Gale-Shapley.

Ce résultat expérimental est cohérent avec notre analyse théorique préliminaire. En effet, la complexité a été établie à $\mathcal{O}(n \times m)$. Cependant, dans le cadre de nos tests, le nombre de parcours m est resté fixe ($m = 10$). La formule théorique devient ainsi $\mathcal{O}(10 \times n)$, ce qui se simplifie en $\mathcal{O}(n)$.

Asymétrie de convergence : l'impact du côté proposant

On remarque que pour $n = 2250$, la variante "côté étudiant" converge en ~ 2500 itérations, contre près de 12000 pour la variante "côté parcours". Bien que les deux algorithmes soient en $\mathcal{O}(n)$, la constante multiplicative est environ 5 fois plus grande lorsque les parcours font les propositions.

Ce phénomène est cohérent avec l'analyse théorique de l'algorithme de Gale-Shapley en présence de capacités asymétriques :

- **Côté Étudiant** : La liste de préférences d'un candidat est bornée par m (un petit nombre, 10 dans notre cas). Le processus de proposition s'épuise donc très rapidement.
- **Côté Parcours** : La liste de préférences d'un master est de taille n . Étant donné qu'un étudiant ne retient qu'une seule offre d'admission (capacité de 1), les masters subissent un taux de rejet massif de la part des meilleurs étudiants. Ils sont contraints de parcourir une grande partie de leur liste de n candidats pour saturer leur capacité c_j , générant ainsi un très grand nombre d'itérations.

Ce constat démontre que pour optimiser le temps de calcul, il est préférable de faire proposer le parti ayant la plus petite liste de préférences.

3 Équité et PL(NE)

3.1 PLNE permettant de déterminer une affectation qui maximise l'utilité minimale des étudiants.

Modélisation de la recherche de l'affectation maximisant l'utilité minimale des étudiants

Variables: Pour tout étudiant $i \in \{0, \dots, 12\}$ et tout parcours $j \in \{0, \dots, 9\}$, on définit la variable x_{ij} telle que $x_{ij} = 1$ si l'étudiant e_i est affecté au parcours p_j et $x_{ij} = 0$ sinon.

Fonction objectif :

$$\max \left(\min_{i \in \{0, \dots, 12\}} \sum_{j=0}^9 x_{ij} u_{e_i}(p_j) \right)$$

La fonction min n'étant pas linéaire, on peut introduire, comme dans le cours, une variable auxiliaire Z représentant l'utilité du plus malheureux des étudiants pour linéariser le problème. Il se reformule comme suit :

Fonction objectif :

$$\max Z$$

Contraintes :

$$\begin{aligned} \sum_{j=0}^9 x_{ij} &= 1, & \forall i \in \{0, \dots, 12\} & \quad (\text{chaque étudiant a un seul parcours}) \\ \sum_{i=0}^{12} x_{ij} &\leq c_j, & \forall j \in \{0, \dots, 9\} & \quad (\text{capacité } c_j \text{ respectée}) \\ Z &\leq \sum_{j=0}^9 x_{ij} u_{e_i}(p_j), & \forall i \in \{0, \dots, 12\} & \quad (Z \text{ est borné par l'utilité de chaque étudiant}) \\ x_{ij} &\in \{0, 1\}, \quad Z \geq 0, & \forall i, j & \quad (\text{domaine des variables}) \end{aligned}$$

Table 2: Matrice $(u_{e_i}(p_j))_{i,j}$ des utilités des étudiants

Etudiants	Parcours									
	0	1	2	3	4	5	6	7	8	9
0	2	1	3	4	0	9	6	7	5	8
1	7	0	4	1	6	8	9	5	2	3
2	8	2	6	3	9	0	1	7	4	5
3	5	1	0	2	3	7	8	6	4	9
4	4	9	3	0	2	5	8	6	1	7
5	9	2	5	3	6	0	1	7	4	8
6	3	2	6	4	0	9	7	8	5	1
7	8	2	6	4	7	0	1	9	5	3
8	3	2	6	4	1	9	7	8	5	0
9	0	3	9	4	2	7	8	1	5	6
10	7	4	1	5	8	3	9	0	6	2
11	0	9	5	4	8	3	2	6	1	7
12	9	1	3	4	0	2	7	6	5	8

3.2 PLNE permettant de trouver une solution qui maximise la somme des utilités (étudiants et parcours)

Modélisation de la recherche de l'affectation maximisant l'utilité totale (efficacité)

Avec les variables x_{ij} définies précédemment, on peut formuler le problème de la manière suivante :

Fonction objectif :

$$\max \sum_{i=0}^{12} \sum_{j=0}^9 x_{ij} (u_{e_i}(p_j) + u_{p_j}(e_i))$$

Contraintes:

$$\sum_{j=0}^9 x_{ij} = 1, \quad \forall i \in \{0, \dots, 12\} \quad (\text{chaque étudiant est affecté à un seul parcours})$$

$$\sum_{i=0}^{12} x_{ij} \leq c_j, \quad \forall j \in \{0, \dots, 9\} \quad (\text{la capacité } c_j \text{ de chaque parcours est respectée})$$

$$x_{ij} \in \{0, 1\}, \quad \forall i, j \quad (\text{domaine des variables})$$

Table 3: Matrice $(U_{ij})_{i,j}$ des utilités totales $u_{e_i}(p_j) + u_{p_j}(e_i)$

Etudiants	Parcours									
	0	1	2	3	4	5	6	7	8	9
0	6	5	6	9	10	20	18	10	16	12
1	12	6	8	7	7	20	19	9	14	8
2	8	2	6	3	9	5	4	7	9	5
3	12	8	12	9	14	17	17	11	14	16
4	12	17	10	8	9	14	15	13	10	8
5	18	12	13	12	12	8	7	15	12	17
6	5	3	11	8	5	16	12	19	12	3
7	20	13	12	15	11	6	5	21	11	14
8	4	4	7	5	4	9	9	9	6	8
9	11	12	19	14	4	10	8	10	9	18
10	10	7	3	8	20	5	10	2	8	5
11	10	14	16	16	16	7	13	12	1	17
12	15	13	12	6	9	3	15	16	8	14

3.3 PLNE maximisant la somme des utilités (étudiants et parcours) parmi les affectations où chaque étudiant a un de ses k premiers choix (pour un k fixé)

Modélisation de la recherche de l'affectation maximisant l'utilité avec garantie de rang

Variables : Celles x_{ij} définies précédemment.

Soit k le rang maximum acceptable pour le vœu d'un étudiant et $m = 10$ le nombre total de parcours.

Fonction objectif : On cherche à maximiser le bonheur total, c'est-à-dire la somme des utilités des étudiants et des parcours pour les affectations réalisées :

$$\max \sum_{i=0}^{12} \sum_{j=0}^9 x_{ij} (u_{e_i}(p_j) + u_{p_j}(e_i))$$

Contraintes :

$$\begin{aligned} \sum_{j=0}^9 x_{ij} &= 1, & \forall i \in \{0, \dots, 12\} & \quad (\text{chaque étudiant a un seul parcours}) \\ \sum_{i=0}^{12} x_{ij} &\leq c_j, & \forall j \in \{0, \dots, 9\} & \quad (\text{la capacité } c_j \text{ est respectée}) \\ \sum_{j=0}^9 x_{ij} u_{e_i}(p_j) &\geq m - k, & \forall i \in \{0, \dots, 12\} & \quad (\text{garantie d'avoir un des } k \text{ premiers choix}) \\ x_{ij} &\in \{0, 1\}, & \forall i, j & \quad (\text{domaine des variables}) \end{aligned}$$

Le plus petit k pour lequel notre PLNE retourne un mariage parfait est $k = 5$. Cela signifie que dans la solution optimale trouvée, chaque étudiant est affecté à un parcours figurant parmi ses 5 premiers choix.

Table 4: Affectations obtenues

Parcours	0	1	2	3	4	5	6	7	8	9
Étudiants	E_7, E_{12}	E_4	E_9	E_{10}	E_2, E_{11}	E_1	E_8	E_6	E_0	E_3, E_5

3.4 Synthèse des approches

Table 5: Comparaison des performances et de la stabilité des différents algorithmes d'affectation.

Algorithme	Utilité Totale	Utilité Moyenne	Utilité Minimale	Paires Instables
GS (Côté Étudiants)	203	7.81	4	0
GS (Côté Parcours)	203	7.81	4	0
PLNE Équitable	192	7.38	5	3
PLNE Utilitariste	213	8.19	4	7
PLNE Top- k ($k = 5$)	201	7.73	5	5

L'analyse de ces résultats nous permet de tirer les conclusions suivantes sur les différents modèles étudiés:

- **Gale-Shapley (Stabilité absolue)**
 - **Force :** Garantit mathématiquement 0 paire instable, assurant une robustesse sociale parfaite (aucune contestation possible).
 - **Faiblesse :** L'utilité totale plafonne à 203. Le système "gâche" du potentiel de satisfaction globale pour empêcher toute instabilité.
- **PLNE Utilitariste (Efficacité maximale)**
 - **Force :** Atteint le maximum théorique de satisfaction globale avec un score de 213.
 - **Faiblesse :** C'est un modèle "injuste" et très instable. Il sacrifie certains étudiants (utilité minimale à 4) pour booster le score global et génère 7 paires instables en forçant des combinaisons purement comptables.
- **PLNE Équitable (Justice sociale)**
 - **Force :** Protège les plus vulnérables en garantissant le 5ème vœu au minimum (utilité minimale de 5).
 - **Faiblesse :** Illustre le "prix de l'équité". Pour sauver un étudiant, le système fait chuter l'utilité totale à 192 et génère tout de même 3 paires instables.
- **PLNE Top- k (Compromis optimal)**
 - **Force :** Avec $k = 5$, il garantit l'équité (personne sous le 5ème vœu) tout en restaurant une excellente utilité totale (201), très proche du score de Gale-Shapley.
 - **Faiblesse :** L'instabilité résiduelle (5 paires instables) est le prix à payer pour maintenir ce niveau de garantie individuelle sans trop dégrader le bien-être collectif.

4 Conclusion

En définitive, ce projet démontre qu'il n'y a pas d'algorithme "magique" ou parfait pour résoudre le problème des mariages stables. Le choix d'une méthode d'affectation relève avant tout d'un choix politique et institutionnel :

- Si l'institution veut éviter à tout prix les contestations étudiantes : il faut choisir l'algorithme de **Gale-Shapley**.
- Si l'institution souhaite maximiser le bonheur global de la promotion avec un filet de sécurité : il faut choisir le **PLNE Top- k** .